



Università degli Studi di Bergamo
Progettazione, Algoritmi e Computabilità

Luca Maccari
Matr. 1081951

Gabriele Masinari
Matr. 1079692

Francesco Tomasoni
Matr. 1080980

Il problema

- ! Code alle casse e comande cartacee soggette a errori
- ⚠ Nessun coordinamento automatico tra reparti (cucina, bar, griglia)
- ⚠ Scorte disallineate rispetto agli ordini reali in tempo reale
- ⚙ Logistica manuale inefficiente per i volontari durante la festa
- Assenza di visibilità in tempo reale sul flusso del servizio

Funzionalità principali

- ✓ POS touch-friendly per composizione e conferma ordini
- Smistamento automatico ai reparti: cucina, bar, pizzeria, griglia
- ≡ Stampa ESC/POS su stampanti termiche via TCP
- ⚠ Gestione scorte con soglie visive e predittore di esaurimento
- ◆ Pannello admin protetto (bcrypt + token) e simulazione real-time

Iterazione

#0

Analisi dei requisiti e architettura del sistema

Attori del sistema



Cassiere

ATTORE PRIMARIO

Operatore alla cassa. Gestisce ordini, pagamenti, note e stampe durante la festa.



Admin

ATTORE PRIMARIO

Configura il sistema prima della festa: festa: menu, stampanti, scorte, allergeni. allergeni.

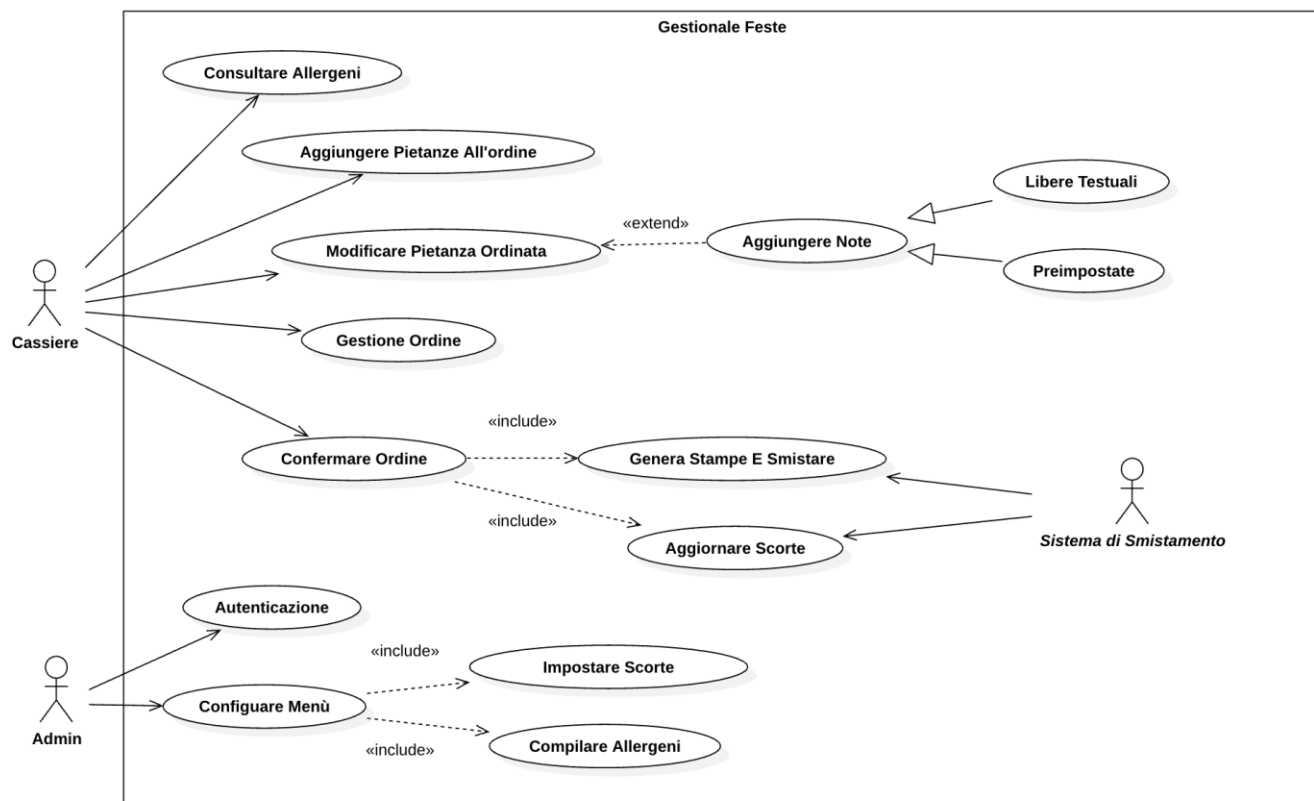


Sist. Smistamento

ATTORE SECONDARIO

Sottosistema che smista e invia i biglietti di stampa alle postazioni dei reparti.

Casi d'Uso



9 CASI D'USO

UC1 Consultare Allergeni

UC2 Aggiungere Pietanze

UC3 Modificare Pietanza

UC4 Gestione Ordine

UC5 Confermare Ordine

UC6 Configurare Menu

UC7 Smistatore Intelligente

UC8 Autenticazione Admin

UC9 Predittore Scorte

9 casi d'uso identificati (UC1 -> UC9), dettagliati nelle Fully Dressed Descriptions del documento di progetto.

Requisiti

REQUISITI FUNZIONALI

- Gestione ordine: un tap aggiunge/incrementa la pietanza
- Scorte limitate: soglie visive (giallo/rosso), blocco automatico
- Gestione stampanti per settore: cucina, bar, griglia, pizzeria
- Conferma ordine: un tasto stampa, aggiorna scorte, azzerà riepilogo
- Autenticazione Admin: password riservata e sessione con scadenza

REQUISITI NON FUNZIONALI

- Efficienza: aggiornamenti scorte/totali in tempo reale
- Usabilità: layout adattivo, note preimpostate, colori tasti tasti
- Affidabilità: coda di stampa robusta anche nei picchi di affluenza
- Configurabilità: soglie, colori e stampanti senza toccare il codice

Architettura del Sistema: 3-Tier

PRESENTATION TIER WebAppCassa (POS) · WebAppBackOffice (Admin)

APPLICATION TIER HTTP/REST + WebSocket

DATA TIER MySQL/MariaDB · Docker · Stampanti ESC/POS
ESC/POS (TCP)

VANTAGGI

Manutenibilità Scalabilità Centralizzazione

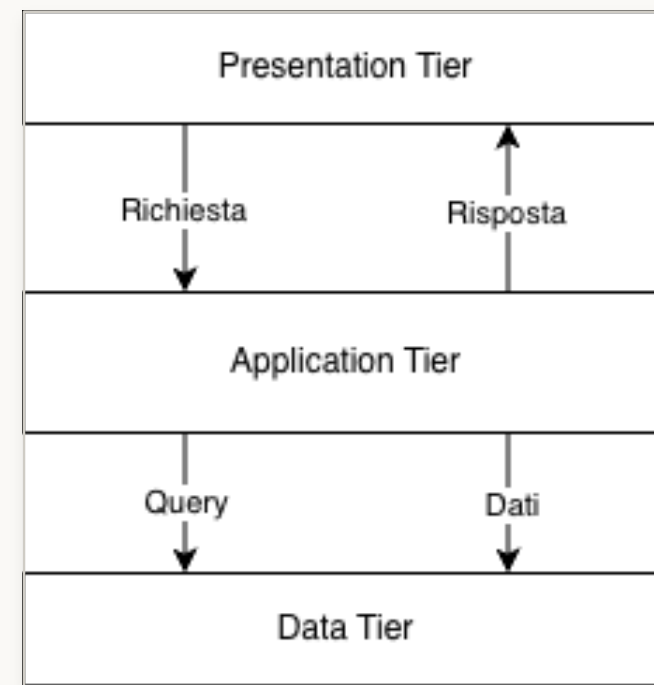


Fig. 1 - Architettura 3-Tier: Presentation, Application, Data layers

Topology Diagram

PRESENTATION TIER

I client, rappresentati dalle casse, comunicano con il server centrale tramite HTTP/REST per le API (es. API (es. Creazione ordine) e il Web Socket per l'aggiornamento delle scorte.

APPLICATION TIER

Il server centrale accede al database relazionale MySQL/MariaDB, con cui comunica tramite MySQL Protocol. Le stampanti di reparto (cucina, bar, pizzeria) ricevono gli ordini direttamente dal server tramite ESC/POS over TCP.

DATA TIER

Garantisce la persistenza di dati.

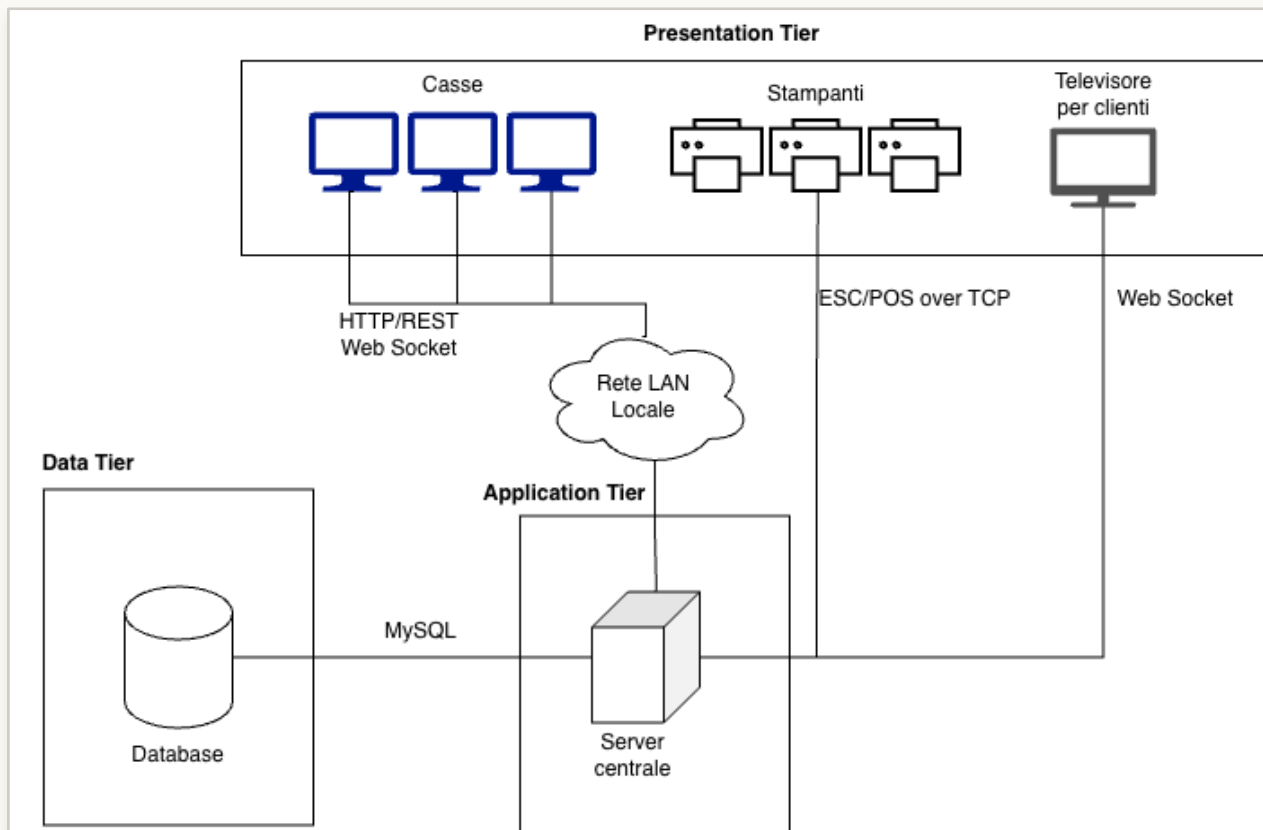


Fig. 2 - Topology: casse, server, stampanti e database su LAN

Stack tecnologico

BACKEND

Node.js / Express 5
MySQL / MariaDB
Socket.io (WebSocket)
ESC/POS via net TCP
Docker + docker-compose

FRONTEND

React (JSX)
Cassa.jsx · Admin.jsx
Hook dedicati
jQuery Ajax (HTTP/REST)

QUALITÀ & TOOL

Vitest + v8 (coverage)
ESLint (complessità ciclomatica)
Madge (accoppiamento)
Postman (API testing)
PlantUML / Draw.io
VS Code · Git/GitHub · LaTeX

Iterazione

#1

Nucleo operativo: ordini, menu, scorte, stampe base

Casi d'uso implementati

CASSIERE

- UC1 - Consultare Allergeni
- UC2 - Aggiungere Pietanze
- UC3 - Modificare Pietanza
- UC4 - Gestione Ordine
- UC5 - Confermare Ordine

ADMIN

- UC6 - Configurare Menu
- UC6a - Impostare Scorte
- UC6b - Compilare Allergeni

Component Diagram

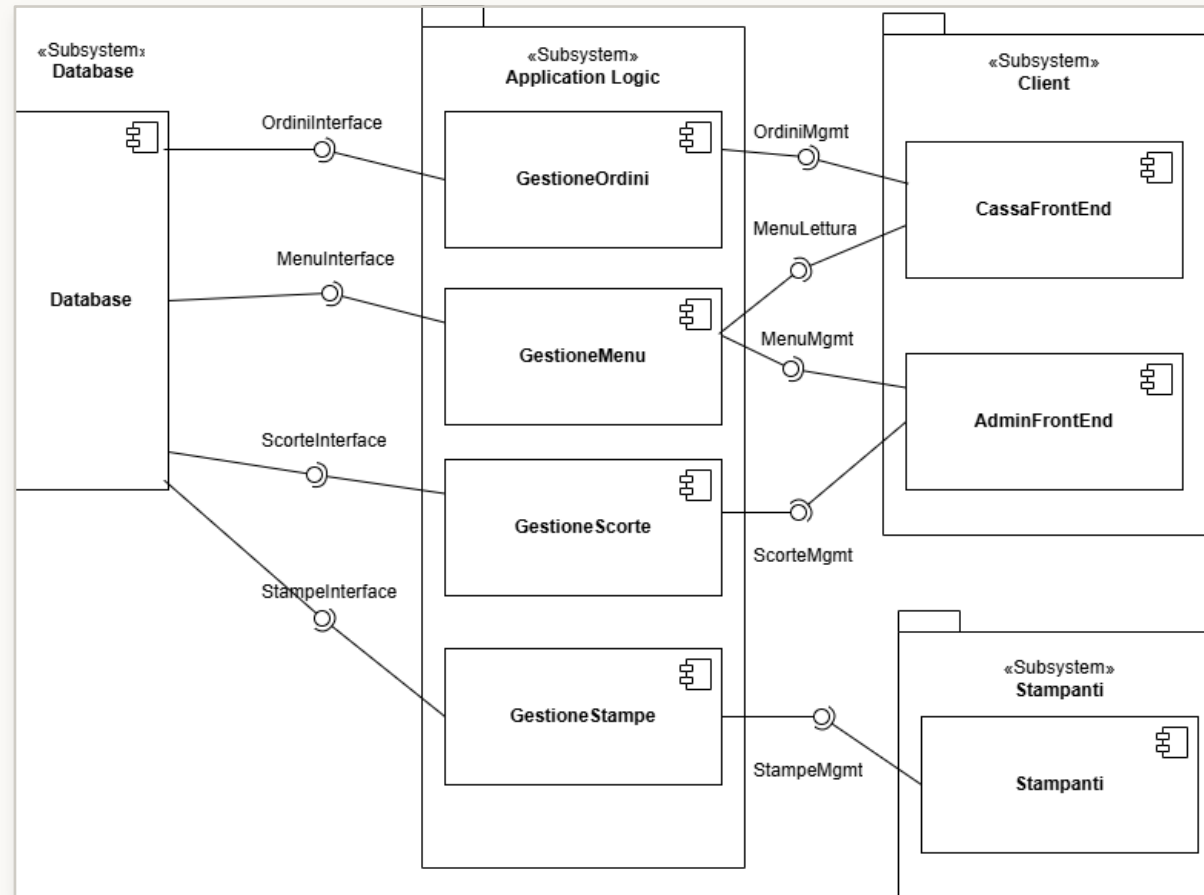


Fig. 3 - Component Diagram Iter.1: Database, Application Logic, Client

Class Diagram - Interfacce

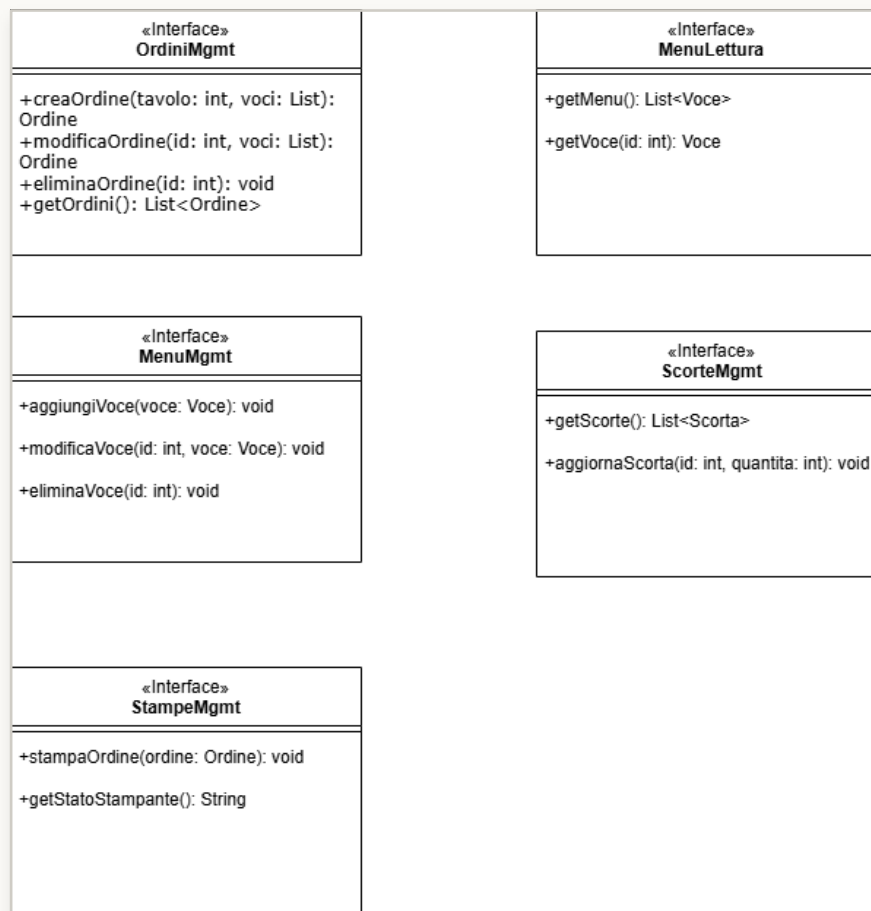


Fig. 4 - Class Diagram Interfacce: contratti tra componenti del sistema

Class Diagram - Tipo di Dato

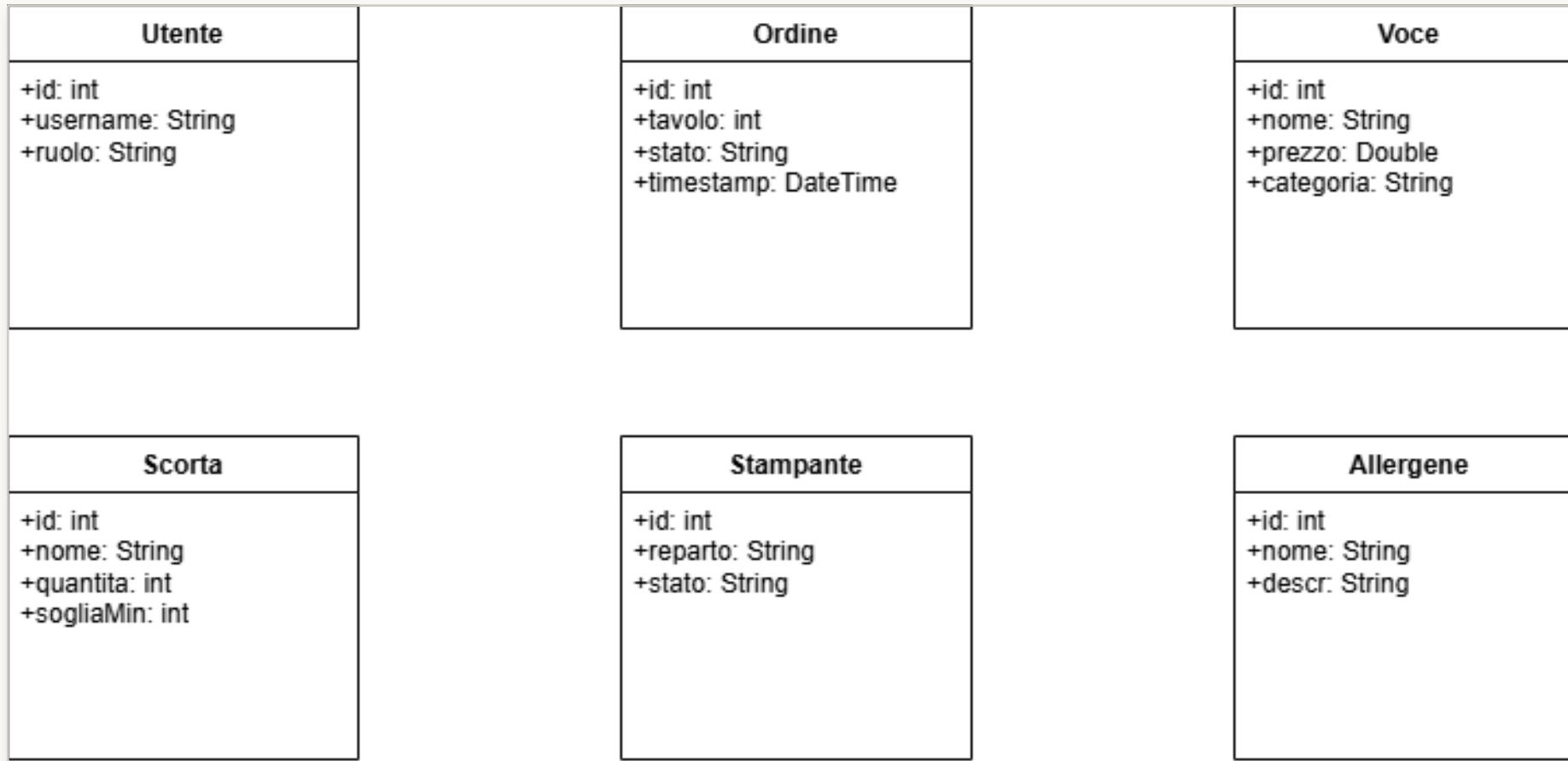


Fig. 5 - Class Diagram Tipi di Dato: strutture dati condivise

Deployment Diagram

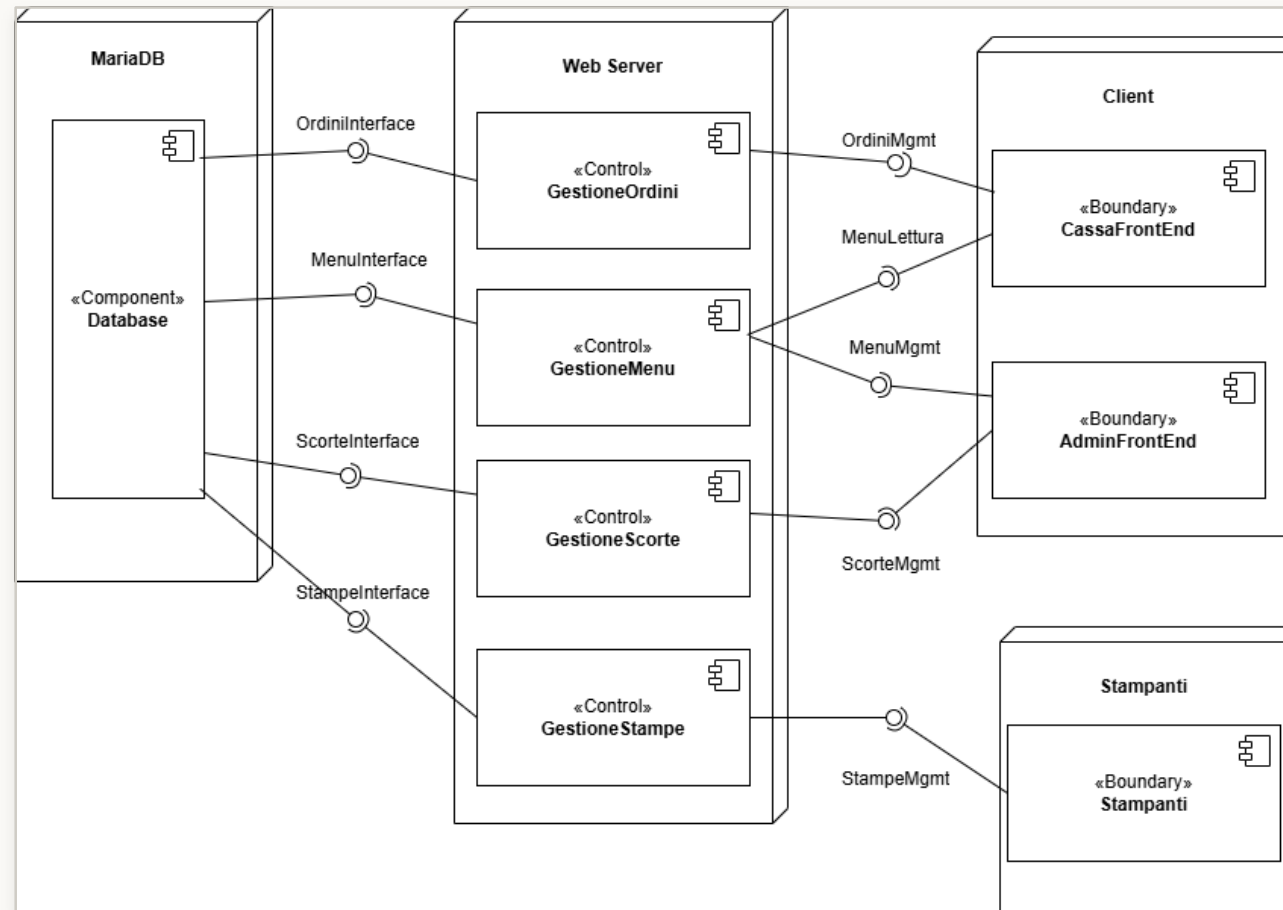


Fig. 6 - Deployment Diagram: mappatura fisica dei componenti sui nodi

Testing

ANALISI STATICA

ESLint - complessità
ciclomantica per moduli
critici

Madge - mapping
accoppiamento e
dipendenze circolari

TEST AUTOMATIZZATI

Vitest con provider v8
Test unitari sui repository
e sui servizi base

API TESTING

Postman- verifica
strutturata degli endpoint
REST

ESLint - Complessità Ciclomatica (CC)

Madge - Accoppiamento e Dipendenze Circolari

Metrica (backend, src/)	Valore
Complessità ciclomatica massima	51 (parsaFlusso)
Funzioni con complessità ciclomatica > 15	4
Funzioni totali analizzate	67
Moduli accoppiati a <code>db.js</code> (C_a)	7
Dipendenze circolari	0
Righe di codice (SLOC, escluse vuote e commenti)	1 506

Fig. 8 - ESLint e Madge: metriche di qualità del codice Iter.1

VITEST-v8

```
C:\Users\masig\Documents\GitHub\Progetto-PAC\CODE\gestionale-feste\backend>npm test

> gestionale-feste-backend@1.0.0 test
> vitest run

v4.1.9 C:/Users/masig/Documents/GitHub/Progetto-PAC/CODE/gestionale-feste/backend

P __tests__/scorte.test.js (10 tests) 46ms
P __tests__/ordini.test.js (12 tests) 66ms
P __tests__/menu.test.js (18 tests) 85ms
P __tests__/stampanti.test.js (7 tests) 20ms

Test Files  4 passed (4)
Tests      47 passed (47)
Start at   17:42:08
Duration   3.77s (transform 193ms, setup 0ms, import 4.72s, tests 218ms, environment 2ms)
```

Fig. 9 - Report Vitest v8: casi di test per Iter.1

API principali e Test con Postman

/api/ordini

GET/api/ordini

POST/api/ordini

POST/api/ordini/:id/conferma

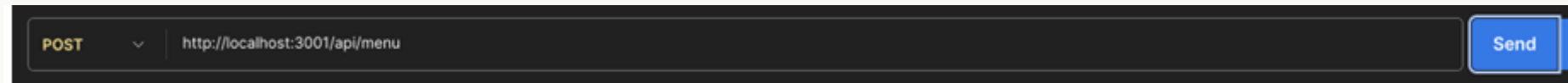


/api/menu

GET/api/menu

POST/api/menu

PATCH/api/menu/:id



/api/scorte · /api/stampanti

GET/api/scorte

PUT/api/scorte/:id

POST /api/stampanti



Iterazione 1 - Cassa (FrontEnd)

BAR	PRIMI	SECONDI	CONTORNI	DOLCI
Acqua 10 € 1.00	Gnocchi al ragu 10 € 8.00	Costine alla griglia 10 € 12.00	Fagioli 10 € 3.00	Crostata 10 € 4.00
Aranciata 10 € 2.50	Lasagne al forno 10 € 8.50	Cotoletta 10 € 9.50	Insalata mista 10 € 3.50	Gelato coppetta 10 € 3.50
Birra media 10 € 3.50	Pasta al pomodoro 10 € 7.00	Pollo arrosto 10 € 10.00	Patatine fritte 10 € 4.00	Panna cotta 10 € 4.50
Birra piccola 10 € 2.50	Penne all'arrabbiata 10 € 7.50	Salsiccia alla piastra 10 € 9.00	Verdure grigliate 10 € 5.00	Tiramisu 10 € 4.50
Cola 10 € 2.50	Polenta taragna 10 € 8.00	Spezzatino 10 € 11.00		
Vino rosso calice 10 € 3.00	Risotto ai funghi 10 € 9.00			

ORDINE

0 prodotti

CRONOLOGIA

VUOTO

Nessuna voce selezionata

AZZERA

SCONTO

ASPORTO

ALLERGENI

Subtotale € 0.00

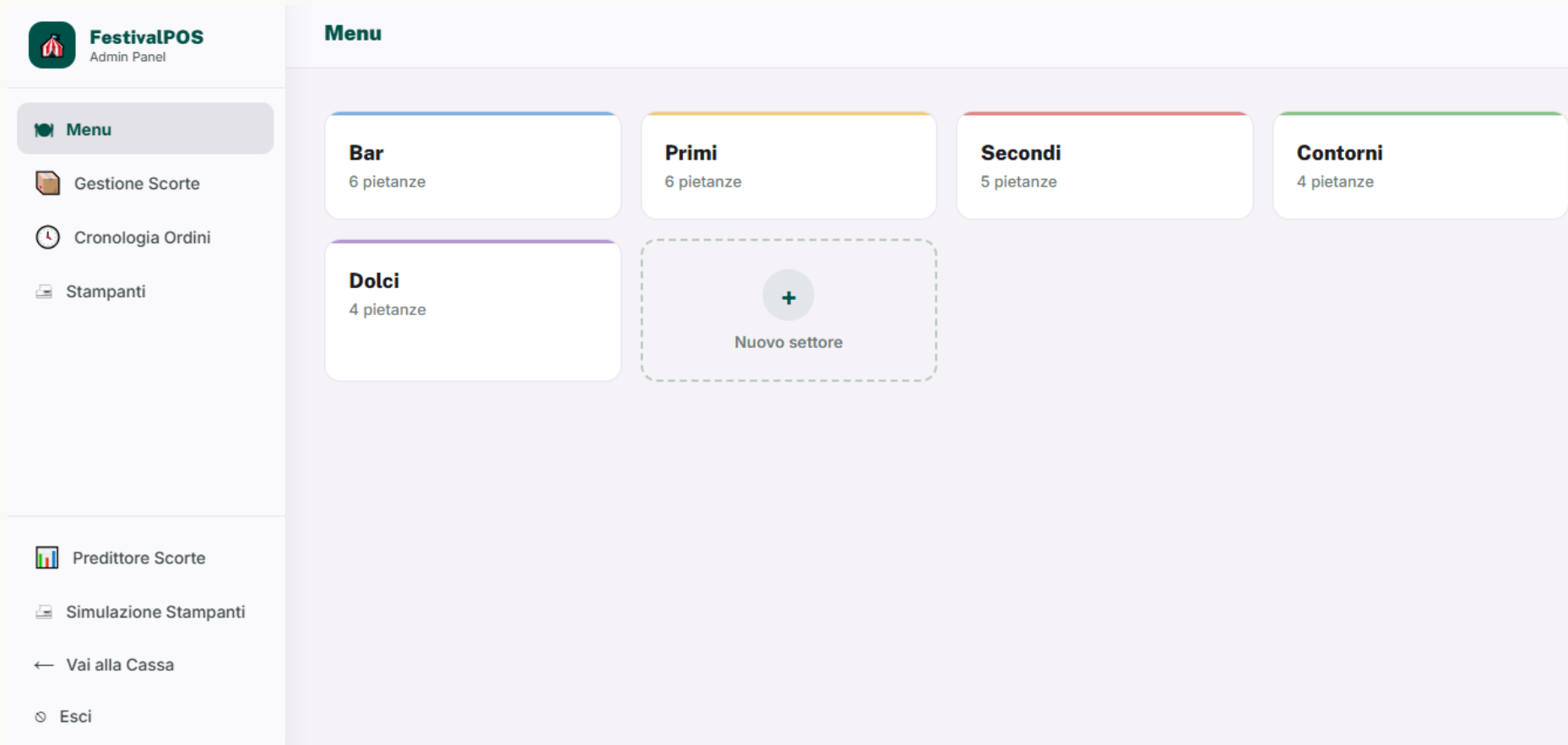
TOTALE € 0.00

Importo pagato (€)

CONFERMA ORDINE ✓

Interfaccia Cassa: composizione ordine a sinistra, riepilogo e conferma a destra

Iterazione 1 - Admin (FrontEnd)



Pannello Admin: navigazione laterale verso Menu, Scorte, Cronologia e Stampanti

Iterazione 1 - Admin (Menu)

FestivalPOS
Admin Panel

Menu

← Settori **Bar** (6 Pietanze) + Nuova pietanza Nuova aggregata

ID	NOME	PREZZO	CATEGORIA	SCORTE	COLORE	VISIBILE	AZIONI
B002	Acqua	1.00 €	Bevanda	10		Visibile	  
B004	Aranciata	2.50 €	Bevanda	10		Visibile	  
B001	Birra media	3.50 €	Bevanda	10		Visibile	  
B006	Birra piccola	2.50 €	Bevanda	10		Visibile	  
B003	Cola	2.50 €	Bevanda	10		Visibile	  
B005	Vino rosso calice	3.00 €	Bevanda	10		Visibile	  

Elenco pietanze del reparto Bar



Nuova Pietanza

NOME PIETANZA *

PREZZO (€) CATEGORIA

SETTORE IN CASSA REPARTO STAMPA

COLORE TASTO



+ Mostra impostazioni avanzate

Annulla Crea Pietanza

Creazione di una nuova pietanza

Iterazione 1 - Admin (Scorte/Ordini)

Menu

Gestione Scorte

Cronologia Ordini

Stampanti

Predittore Scorte

Simulazione Stampanti

Vai alla Cassa

Gestione Scorte

ARTICOLO	QUANTITÀ	SOGLIA GIALLO	SOGLIA ROSSO	STATO GIALLO	STATO CRITICO	AZIONI
Birra media	10	10	3	⚠	—	Rifornisci
Acqua	0	10	3	—	🔴	Rifornisci
Cola	10	10	3	⚠	—	Rifornisci
Aranciata	10	10	3	⚠	—	Rifornisci
Vino rosso calice	10	10	3	⚠	—	Rifornisci
Birra piccola	10	10	3	⚠	—	Rifornisci
Patatine fritte	9	10	3	⚠	—	Rifornisci
Insalata mista	9	10	3	⚠	—	Rifornisci

Gestione scorte con soglie

Menu

Gestione Scorte

Cronologia Ordini

Stampanti

Predittore Scorte

Simulazione Stampanti

Vai alla Cassa

Esci

Cronologia Ordini

Cronologia Ordini (1)

Ordine #1

03/07/2026, 16:58:40

28.00 €
5 articoli

Articoli:

Acqua × 1
Prezzo: 1.00 €

Gnocchi al ragu × 1
Prezzo: 8.00 €

Costine alla griglia × 1
Prezzo: 12.00 €

Fagioli × 1
Prezzo: 3.00 €

Crostata × 1
Prezzo: 4.00 €

Totale lordo: 28.00 €

Totale: 28.00 €

Cronologia ordini confermati

Iterazione

#2

Smistatore Intelligente Multi-Reparto + WebSocket real-time

Casi d'uso e nuove funzionalità

USE CASE

- UC5a - Genera Stampe e Smistare (avanzato)
- UC7 - Configurare Stampanti di Reparto
- Aggiornamento diagrammi UML (Component, Class, Deployment)
- UC6b - Compilare allergeni

REAL-TIME

- ordine_creato - broadcast nuovi ordini
- scorta_aggiornata - alert visivi cassiere
- stampante_stato - notifiche offline/online
- Socket.io con room per ruolo

Component Diagram

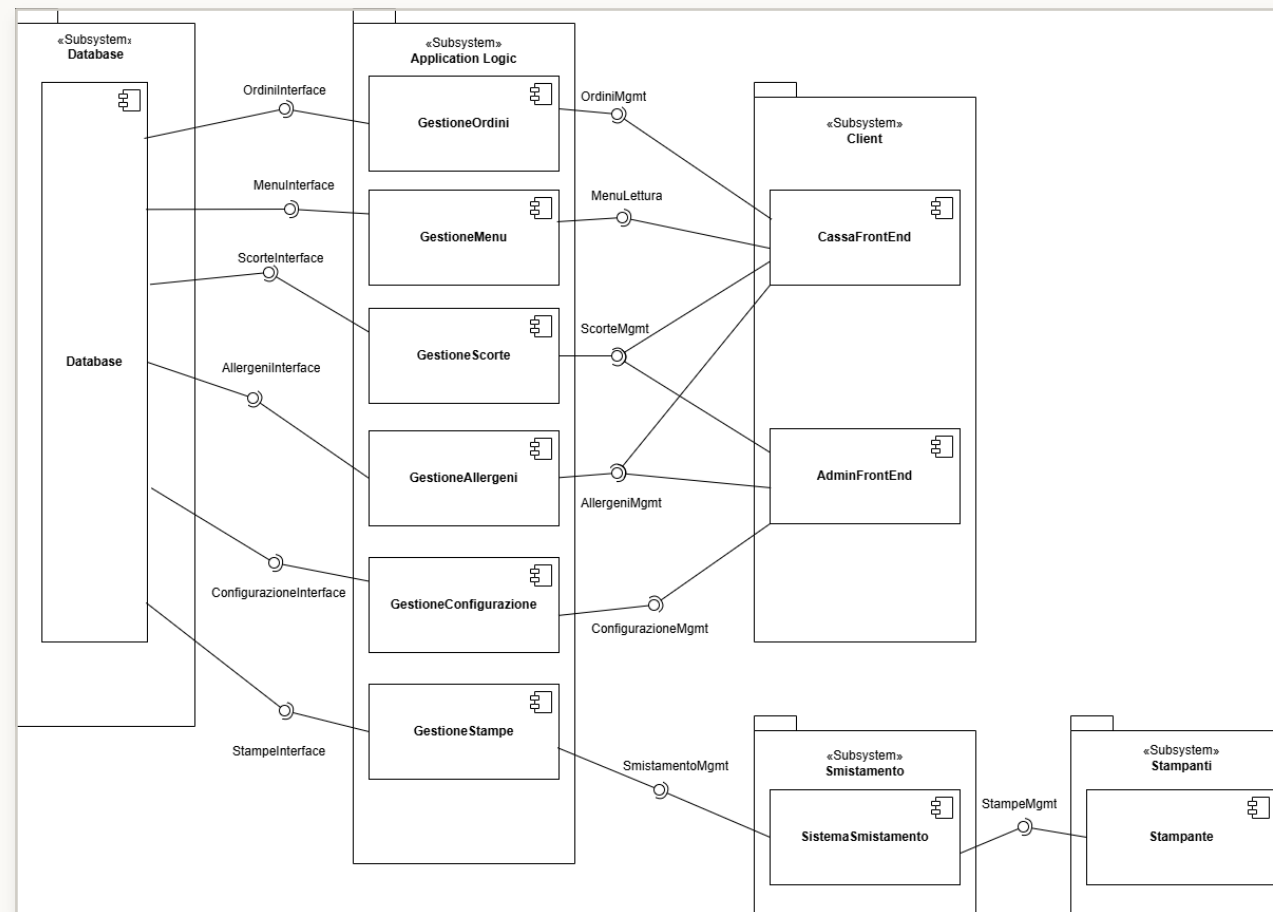


Fig. 10 - Component Diagram Iter.2: aggiunta dei componenti Allergeni, Configurazione e Smistamento

Class Diagram - Interfacce

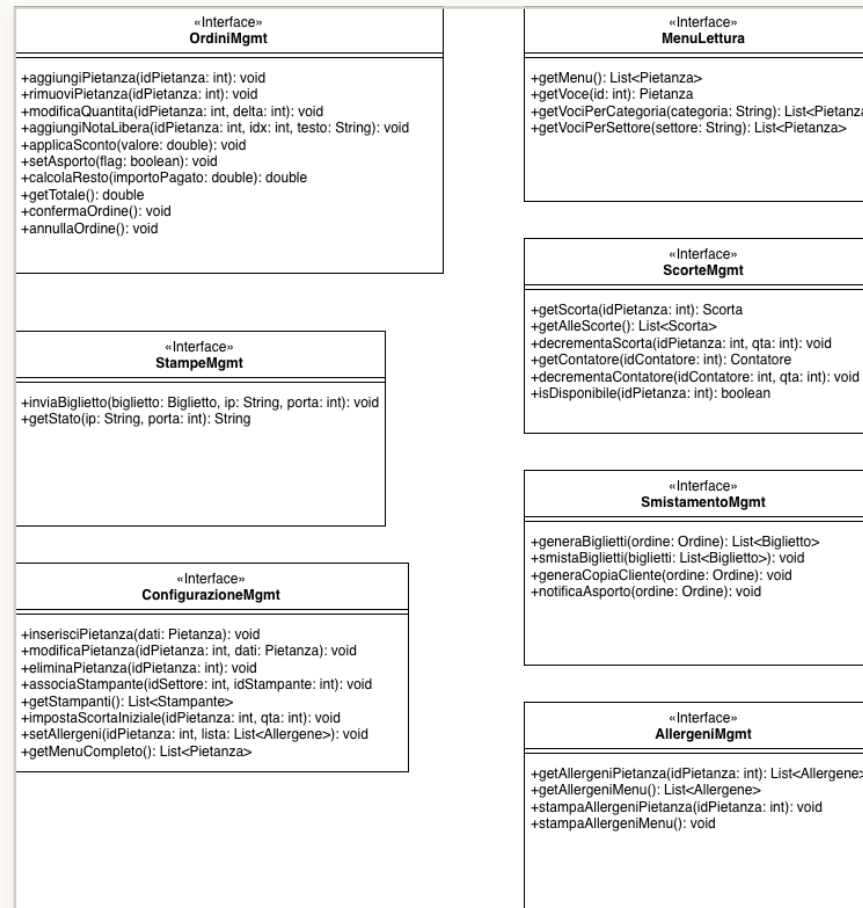


Fig. 11 - Class Diagram Interfacce Iter.2: Smistatore e WebSocket

Class Diagram - Tipo di Dato

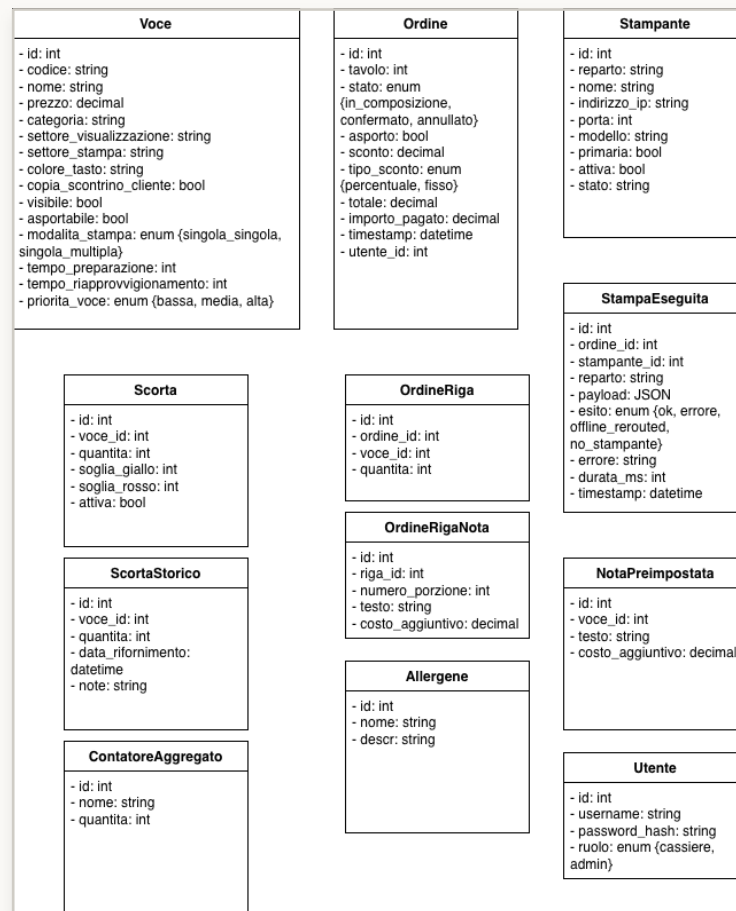


Fig. 12 - Class Diagram Dati Iter.2: entit  smistamento e real-time

Deployment Diagram

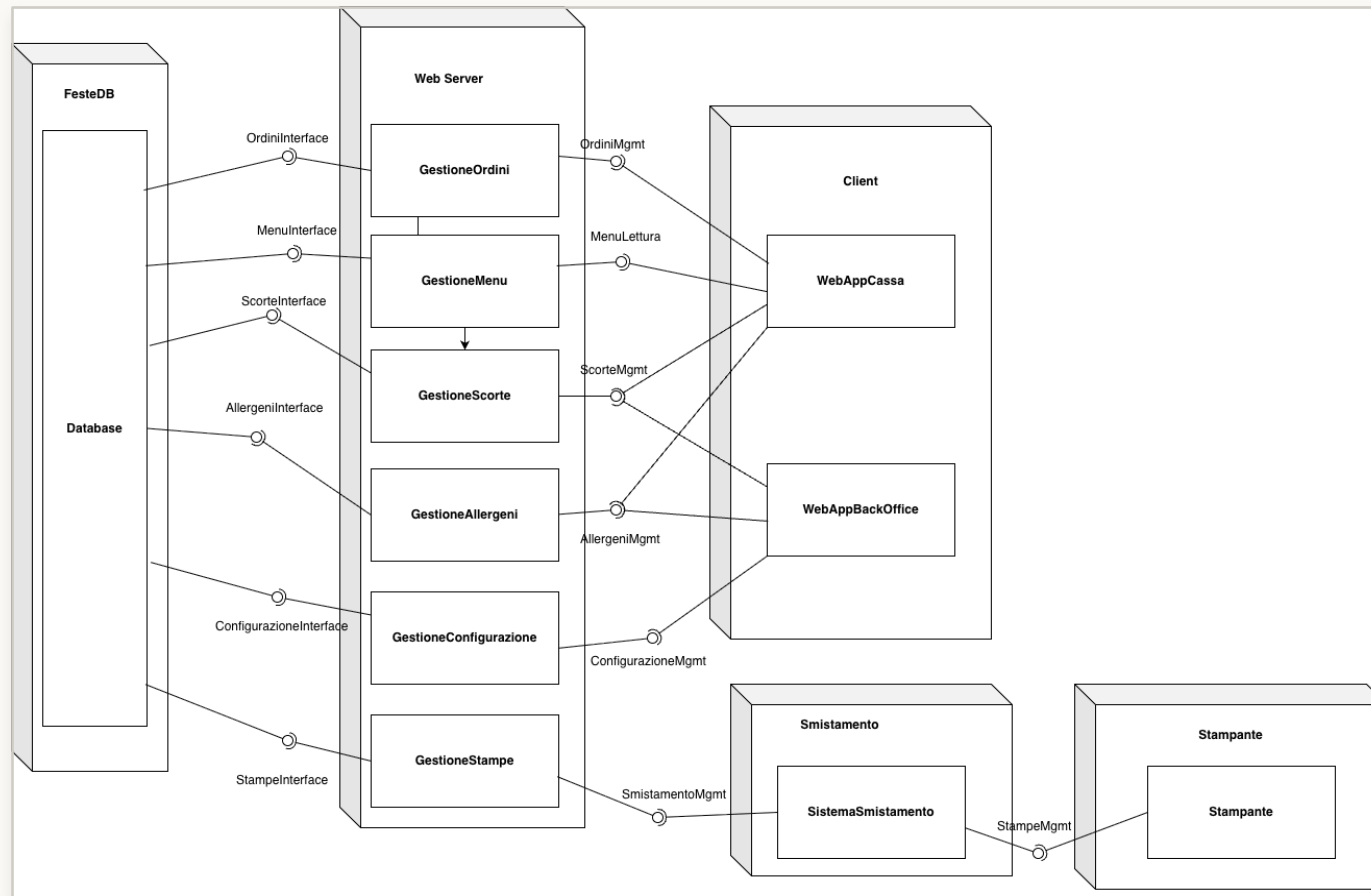


Fig. 13 - Deployment Diagram Iter.2: nodi fisici e WebSocket

Smistatore Intelligente

1

Scoring del carico per reparto

carico = $n_task \times t_prep_residuo \times stress \times$
stampante, per ogni reparto compatibile con la
pietanza

2

Assegnazione al carico minimo

selezione greedy del reparto con carico piu
basso; fallback a FIFO se un solo reparto e
disponibile

3

Ranking P in coda

priorita = attesa + asporto + semplicita - carico -
rischio scorte; ordina le comande nel reparto

$O(n \cdot m + k \log k)$

```

1: procedure ROUTEORDER(order)
2:   tasks ← []
3:   for riga ∈ order.righe do                                ▷ O(k)
4:     task ← BUILDTASK(riga, order.timestamp, order.priorità)
5:     tasks.append(task)
6:   end for
7:   for task ∈ tasks do                                       ▷ O(k)
8:     if ¬SCORTEDISPONIBILI(task) then
9:       BLOCCA(task); NOTIFICACASSA(task)
10:      tasks.remove(task)
11:    end if
12:  end for
13:  comande ← {}
14:  for task ∈ tasks do                                       ▷ O(k · n · m)
15:    reparti ← GETREPARTICOMPATIBILI(task)                  ▷ O(n)
16:    r ← SELECTMINCARICO(reparti)                            ▷ O(n · m)
17:    task.P ← CALCOLARANKING(task, r)                        ▷ O(1)
18:    comande[r].append(task)
19:  end for
20:  comande ← RAGGRUPPAEBATCH(comande)                       ▷ O(k log k)
21:  for r ∈ KEYS(comande) do
22:    task_list ← SORTBYP(comande[r])                         ▷ O(k log k)
23:    comanda ← GENERACOMANDA(r, task_list)
24:    INVIACOMANDA(comanda, r)
25:  end for
26: end procedure

```

Smistatore - Scoring e Ranking

Per ogni riga d'ordine, l'algoritmo calcola un **punteggio di carico** per ciascun reparto compatibile e assegna il task al reparto con carico minimo.

Una volta assegnato il reparto, il ranking P determina l'**ordine di elaborazione** nella coda (valore più alto = priorità maggiore):

```

1: procedure SELECTMINCARICO(reparti) ▷  $O(n \cdot m)$ 
2:   best ← null; best_score ←  $+\infty$ 
3:   for  $r \in \text{reparti}$  do
4:     score ←  $\sum_{\text{task} \in r.\text{coda}} \text{task}.t_{\text{prep\_residuo}}$ 
5:     score ← score  $\times \alpha_{\text{stress}}(r) \times \beta_{\text{stampante}}(r)$ 
6:     if score < best_score then
7:       best ←  $r$ ; best_score ← score
8:     end if
9:   end for
10:  return best
11: end procedure

12: procedure CALCOLARANKING(task, r) ▷  $O(1)$ 
13:   $t_{\text{attesa}} \leftarrow \text{NOW} - \text{task}.t_{\text{timestamp}}$ 
14:   $\pi_a \leftarrow [\text{task}.priorità = \text{ASPORTO}] ? 1.0 : 0.0$ 
15:   $s \leftarrow 1.0 / (\text{task}.t_{\text{prep}} + 1)$ 
16:  return  $A \cdot t_{\text{attesa}} + B \cdot \pi_a + C \cdot s - D \cdot \text{carico}(r) - E \cdot \text{rischio\_scorte}(\text{task})$ 
17: end procedure

```

Fig. 14 - Pseudocodice RouteOrder: scoring per carico minimo e ranking P

API Smistatore - Test con Postman

REST - /api/smistatore

GET/api/smistatore/stato

PUT/api/smistatore/stampante/:reparto

PUT/api/smistatore/fallback/:reparto

POST/api/smistatore/completa

GET/api/stampe



ESLint - Complessità Ciclomatica (CC)

Madge - Accoppiamento e Dipendenze Circolari

Metrica (backend, src/)	Base	Con Smistatore	Δ
Complessità ciclomatica massima	51	51	=
Funzioni con complessità ciclomatica > 15	4	4	=
Funzioni totali analizzate	67	85	+18
Moduli accoppiati a <code>db.js</code> (C_a)	7	8	+1
Dipendenze circolari	0	0	=
Righe di codice (SLOC, escluse vuote e commenti)	1 506	1 732	+226

Fig. 15 - ESLint e Madge Iter.2: complessità ciclomatica e dipendenze aggiornate

Vitest v8 - Smistatore Intelligente

```
✓ __tests__/smistatore.test.js (26 tests) 11ms
✓ Formula del carico (5)
  ✓ reparto vuoto → carico 0 1ms
  ✓ 1 task tprep=60 → carico = 60 0ms
  ✓ 5 task tprep=60 →  $\alpha=1.3$  ATTIVO → carico = 1950 0ms
  ✓ 4 task tprep=60 →  $\alpha$  NON attivo → carico =  $4 \cdot 240 = 960$  0ms
  ✓ stampante offline →  $\beta = 2.0$  0ms
✓ selectMinCarico (arg-min + tie-break) (3)
  ✓ sceglie il reparto con carico minore 0ms
  ✓ tie-break: a parità di carico, sceglie il reparto col last-task più vecchio 5ms
  ✓ tutti offline → ritorna comunque il primo candidato 0ms
✓ Routing (routeOrder) (7)
  ✓ riga con scorta sufficiente → 1 task generato 1ms
  ✓ riga con scorta INSUFFICIENTE → finisce in bloccati, NON in comande 0ms
  ✓ quantita=3 + singola_multipla → 1 task con quantita=3 0ms
  ✓ quantita=3 + singola_singola → 3 task in coda, batchati in 1 comanda qta=3 0ms
  ✓ batching: due voci diverse, stesso reparto → 1 comanda con 2 righe 0ms
  ✓ smista su più reparti e ordina le comande per priorità P decrescente 0ms
  ✓ applica i fallback ai default quando mancano settore_stampa e tempo_preparazione 0ms
✓ Ranking P (3)
  ✓ asporto incrementa P di B=50 rispetto a non-asporto 0ms
  ✓ rischio scorte (sotto soglia rosso) penalizza P di E=30 0ms
  ✓ calcolaRanking applica default se tempo_preparazione è mancante o 0 0ms
✓ Bilanciamento end-to-end con fallback bidirezionale (4)
  ✓ 6 ordini cucina con fallback → cucina_2 → distribuzione 3-3 0ms
  ✓ primario offline + fallback online → routing va al fallback 0ms
  ✓ tutti offline → routing forzato sul primario 0ms
  ✓ imposta i fallback a [] se vengono passati parametri non array 0ms
✓ completaTask (2)
  ✓ rimuove un task dalla coda del reparto 0ms
  ✓ ritorna false se il task non esiste 0ms
✓ snapshot (2)
  ✓ ritorna lo stato corrente delle code 0ms
  ✓ gestisce lo snapshot con stampante offline 0ms
```

Fig. 16 - Report Vitest Smistatore: test di scoring, ranking e load balancing

Iterazione 2 - Smistatore Intelligente

Simulazione StampantiAggiornaTorna alla Cassa

CUCINA A ONLINE
cucina • 127.0.0.1:9100 • EPSON
SPEGNI TEST PULISCI
17:00:59
REPARTO CUCINA

Ordine #3 15:00:59

1x Pollo arrosto

** FINE COMANDA **
%<

%<

STAMPANTE BAR ONLINE
bar • 127.0.0.1:9101 • EPSON
SPEGNI TEST PULISCI
17:00:59
REPARTO BAR

Ordine #3 15:00:59

1x Acqua
1x Gelato coppetta

** FINE COMANDA **
%<

%<

STAMPANTE GRIGLIA ONLINE
griglia • 127.0.0.1:9102 • EPSON
SPEGNI TEST PULISCI
Nessuno scontrino
ancora stampato

CUCINA B ONLINE
cucina_2 • 127.0.0.1:9103 • EPSON
SPEGNI TEST PULISCI
17:00:59
REPARTO CUCINA_2

Ordine #3 15:00:59

1x Pasta al pomodoro
1x Insalata mista

** FINE COMANDA **
%<

%<

STAMPANTE CASSA ONLINE
cassa • 127.0.0.1:9104 • EPSON
SPEGNI TEST PULISCI
17:00:59
COPIA CLIENTE

Ordine #3 15:00:59

1x Acqua
1.00 EUR = 1.00 EUR
1x Gelato coppetta
3.50 EUR = 3.50 EUR

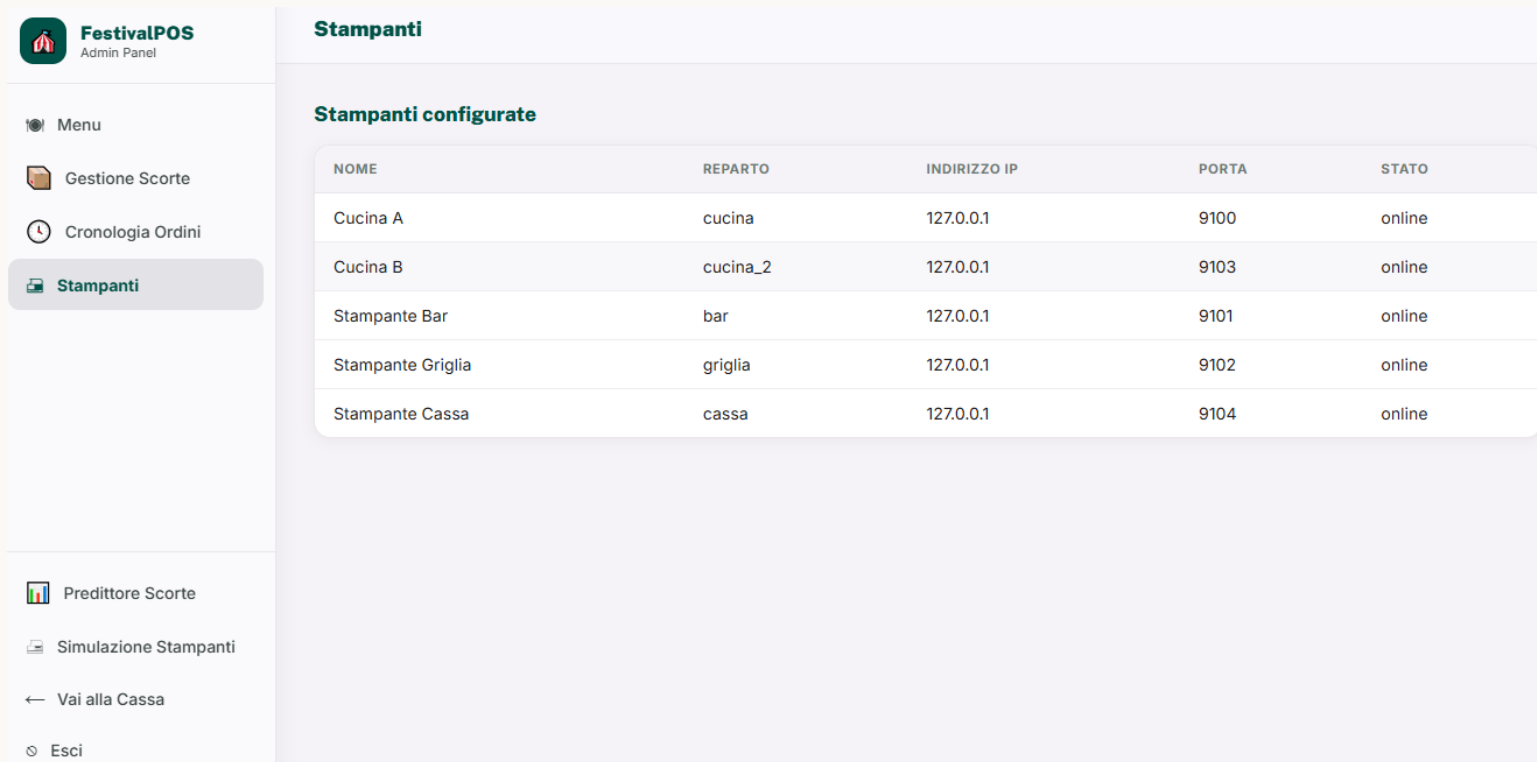
TOTALE: 25.00 EUR
PAGATO: 30.00 EUR
RESTO: 5.00 EUR

** FINE SCONTRINO **
%<

%<

Dashboard Smistatore: stato reparti, code di stampa e fallback in tempo reale

Iterazione 2 - Stampanti



FestivalPOS
Admin Panel

- Menu
- Gestione Scorte
- Cronologia Ordini
- Stampanti**

Predittore Scorte

Simulazione Stampanti

← Vai alla Cassa

Esci

Stampanti

Stampanti configurate

NOME	REPARTO	INDIRIZZO IP	PORTA	STATO
Cucina A	cucina	127.0.0.1	9100	online
Cucina B	cucina_2	127.0.0.1	9103	online
Stampante Bar	bar	127.0.0.1	9101	online
Stampante Griglia	griglia	127.0.0.1	9102	online
Stampante Cassa	cassa	127.0.0.1	9104	online

Dashboard Stampanti Configurate: cucina, bar, griglia, cassa

Iterazione

#3

Predittore Dinamico di Riordino Scorte + Auth Admin + Refactoring

Casi d'uso e refactoring

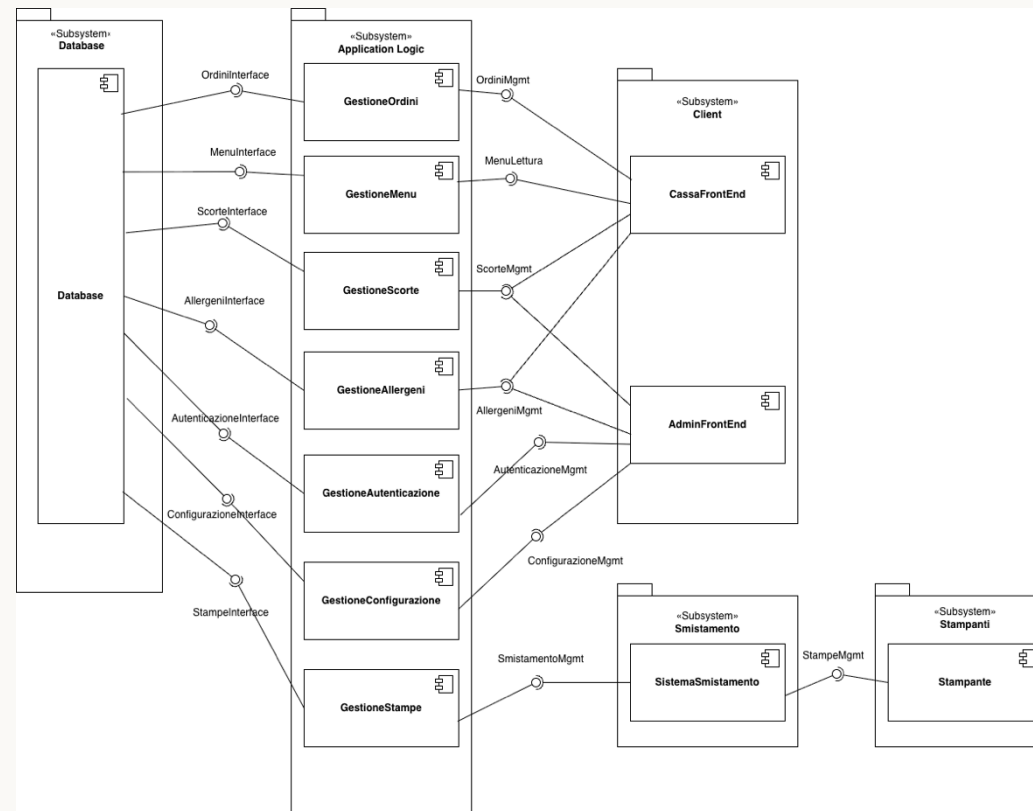
NUOVI CASI D'USO

- UC8 - Autenticazione Admin (PIN bcrypt + token)
- UC9 - Predittore Dinamico di Riordino Scorte

REFACTORING

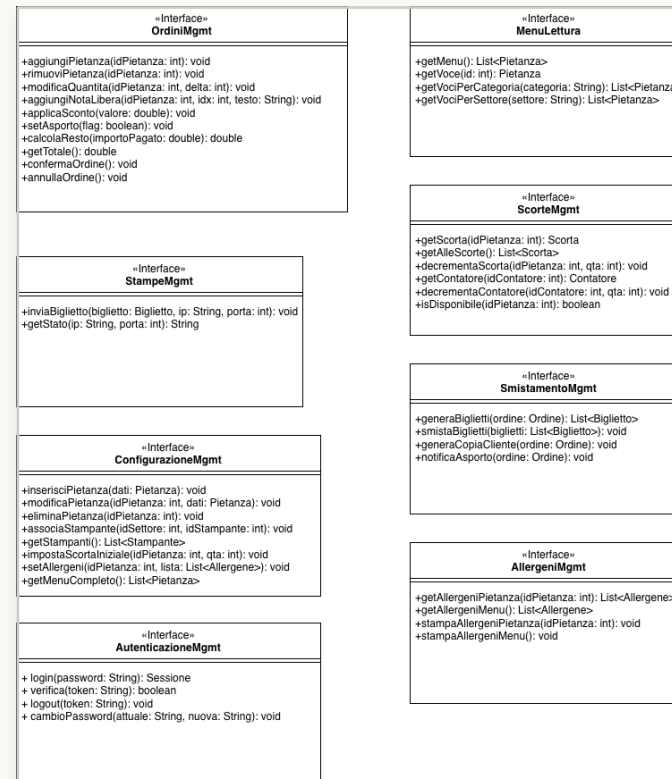
- Decomposizione God Component frontend
- Riduzione accoppiamento (Madge)
- Separazione accesso dati in repository dedicati
- Complessità ciclomatica verificata con ESLint

Component Diagram - Autenticazione



Nuovo componente GestioneAutenticazione nell'Application Logic, con interfaccia AutenticazioneMgmt verso AdminFrontEnd

Class Diagram - Interfaccia AutenticazioneMgmt



Deployment Diagram e Class Diagram per tipo di dato restano invariati rispetto all'Iterazione 2: il Predittore e l'Autenticazione
l'Autenticazione sono servizi interni al Web Server (doc §4.3)

Predittore - Consumo Atteso

Il consumo atteso è una **media pesata** a tre componenti: comportamento recente, trend a medio termine e baseline storica per fascia oraria.

$$\text{consumo_atteso} = \alpha \cdot c_{15\text{min}} + \beta \cdot c_{1\text{h}} + \gamma \cdot \text{media_storica_fascia}$$
$$\alpha + \beta + \gamma = 1$$

NORMALE $\alpha = 0.5, \beta = 0.3, \gamma = 0.2$

PICCO $\alpha = 0.8$ (quando $c_{15\text{min}} > 2 \times \text{media storica}$)

PRUDENTE $\gamma = 0$ se storico < 3 occorrenze (fallback statico)

$\text{tempo_esaurimento} = \text{quantita_attuale} / \text{consumo_atteso}$

Predittore - Indice di Urgenza (U)

L'indice U quantifica il **rischio di esaurimento scorte** prima del completamento del rifornimento. Se $U > 0$ scatta il suggerimento proattivo.

$$U = \max(0, t_{\text{riapprovvigionamento}} - t_{\text{esaurimento}}) \times \text{priorità_voce}$$

PASSI PRINCIPALI

- Raccolta dati: scorte, contatori vendite 15min/1h, fascia oraria
- Aggregazione ingredienti condivisi tra più pietanze
- Se $U > \text{soglia_warn}$: suggerimento rifornimento con quantità consigliata
- Trigger: periodico ogni T minuti o event-driven ad ogni vendita

Predittore - Diagramma di Flusso

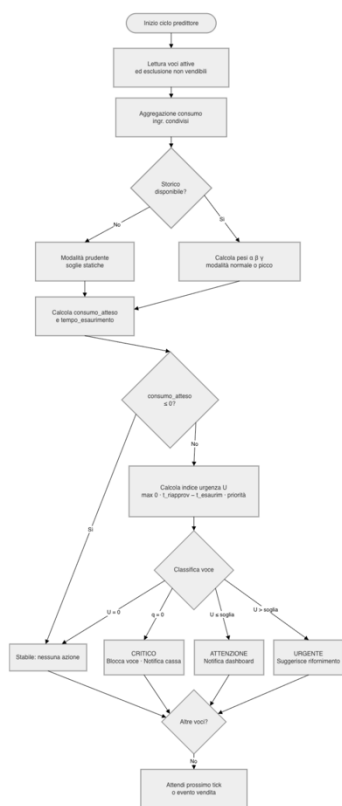


Figura 36: Diagramma di flusso Predittore Dinamico di Riordino Scorte

- Lettura voci attive ed esclusione delle non vendibili
- Aggregazione del consumo per ingredienti condivisi
- Storico disponibile? Sì -> calcola pesi $\alpha/\beta/\gamma$ (normale o picco); No -> modalità modalità prudente (soglie statiche)
- Calcolo di consumo_atteso e tempo_esaurimento
- consumo_atteso ≤ 0 ? Sì -> voce stabile; No -> calcola indice di urgenza U urgenza U
- Classificazione della voce: stabile, attenzione, critico o urgente, con azione associata (notifica dashboard, blocco cassa, suggerimento rifornimento)

API - Predittore e Autenticazione

/api/predittore

GET/api/predittore/scorte

GET/api/predittore/scorte/:id

GET/api/predittore/configurazione

PUT/api/predittore/configurazione/:chiave

PUT/api/predittore/voce/:id

/api/auth/admin

POST/login

GET/verifica

POST/logout

PUT/password

ESLint & Madge - Metriche di Refactoring (Tabella 16)

Metrica	Prima	Dopo
Funzioni con complessità ciclomatica > 15	11	5
Complessità ciclomatica massima	55	37
Moduli accoppiati al DB (C_a di <code>db.js</code>)	10	2
Dipendenze circolari	0	0
Modulo più grande (righe di codice)	1328	331
Numero di moduli	19	50

Fig. 17 - Tabella 16: metriche ESLint e Madge prima/dopo il refactoring

Refactoring - Metriche Prima/Dopo

51 → 11

complessità ciclomatica parseFlusso (CC)

21 → 2

complessità ciclomatica eseguiPredizione (CC)

669 → 227

righe di codice Cassa.jsx

10 → 2

moduli dip. da db.js (repository)

Sottosistema di stampa incapsulato dietro una façade (services/stampa/): le route dipendono ora da un solo modulo invece di quattro servizi distinti.

Copertura finale - Suite Vitest

```
Coverage enabled with v8

_ _tests_/predittore.test.js (33 tests) 50ms
_ _tests_/predittore-integrazione.test.js (7 tests) 310ms
_ _tests_/smistatore.test.js (26 tests) 40ms
_ _tests_/smistatore-integrazione.test.js (5 tests) 4023ms
  [ ] Integrazione HTTP smistatore > GET /api/stampanti restituisce almeno 4 stampanti 920ms
  [ ] Integrazione HTTP smistatore > Ordine multi-reparto → comande corrette + audit OK 1355ms
  [ ] Integrazione HTTP smistatore > Stampante spenta + ordine cucina → routing automatico su cucina_2 1089ms
  [ ] Integrazione HTTP smistatore > Ordine con asporto → comanda con P più alto rispetto a ordine tavolo 323ms
  [ ] Integrazione HTTP smistatore > Scorta esaurita → riga in bloccati, le altre proseguono 332ms

Test Files 4 passed (4)
Tests 71 passed (71)
Start at 14:56:15
Duration 5.00s (transform 173ms, setup 0ms, collect 780ms, tests 4.42s, environment 2ms, prepare 1.20s)

x Coverage report from v8
-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files | 100     | 94.4     | 100     | 100     |
predittore.js | 100     | 93.75    | 100     | 100     | 43,70-71,236,269
smistatore.js | 100     | 95.06    | 100     | 100     | 34,148,176-177
-----
```

12 test di integrazione HTTP verificano l'intera catena: router Express → service → repository → DB → emulatore stampante.

Autenticazione Admin - Test Vitest

```
✓ __tests__/auth.test.js (15 tests) 510ms
  ✓ auth.service - password (5)
    ✓ hashPassword produce un hash bcrypt verificabile 68ms
    ✓ verificaPassword ritorna true con la password corretta 130ms
    ✓ verificaPassword ritorna false con password errata 184ms
    ✓ verificaPassword ritorna false (senza lanciare) con hash assente o malformato 0ms
    ✓ accetta la password anche se passata come numero 123ms
  ✓ auth.service - token di sessione (5)
    ✓ creaToken restituisce un token e una scadenza futura 1ms
    ✓ validaToken riconosce un token valido e ne legge il ruolo 0ms
    ✓ validaToken ritorna null per token inesistente o assente 0ms
    ✓ validaToken ritorna null per token scaduto e lo rimuove 0ms
    ✓ revocaToken invalida il token (logout) 0ms
  ✓ auth.service - estraiToken (2)
    ✓ estrae il token dall'header Authorization Bearer 0ms
    ✓ ritorna null in assenza di header o con formato errato 0ms
  ✓ auth.service - middleware richiediAdmin (3)
    ✓ chiama next() e popola req.utente con un token admin valido 0ms
    ✓ risponde 401 senza token 0ms
    ✓ risponde 401 con un token di ruolo non admin 0ms
✓ __tests__/auth-integrazione.test.js (8 tests) 843ms
```

33/33 test unitari superati - hash bcrypt, token di sessione e middleware richiediAdmin

Predittore Dinamico - Test Vitest

```

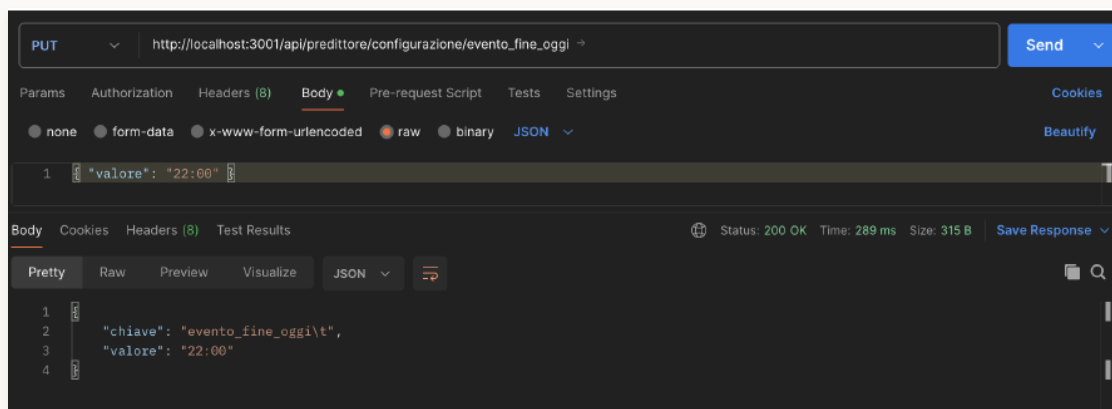
✓ __tests__/predittore.test.js (33 tests) 6ms
✓ calcolaPesi: selezione modalità (4)
  ✓ storico sufficiente + consumo normale → modalità "normale" (α=0.5) 1ms
  ✓ storico sufficiente + consumo a picco (>2× storico) → modalità "picco" (α=0.8) 0ms
  ✓ occorrenze storiche < 3 → modalità "prudente" (γ=0) 0ms
  ✓ storico = 0 → niente picco anche con consumo alto (no divisione su 0) 0ms
✓ consumoAtteso: applica formula (2)
  ✓ α·c15/15 + β·c1h/60 + γ·storico/60 0ms
  ✓ zero consumo ovunque → 0 0ms
✓ tempoEsaurimentoSec (3)
  ✓ quantita=10 + consumo=1 unità/min → 600 secondi 0ms
  ✓ consumo ≤ 0 → Infinity 0ms
  ✓ quantita=0 → 0 secondi 0ms
✓ calcolaU: indice di urgenza (5)
  ✓ tempo_esaur > tempo_riapp → U = 0 (nessuna urgenza) 0ms
  ✓ tempo_esaur < tempo_riapp con priorità media → U = (riapp - esaur) × 1.0 × 1.2 0ms
  ✓ priorità alta moltiplica U per 1.5 0ms
  ✓ priorità bassa moltiplica U per 0.5 0ms
  ✓ moltiplicatore contesto NON attivo se esaur > residuo evento 0ms
✓ classifica: 4 stati (4)
  ✓ quantita = 0 → "esaurito" 0ms
  ✓ U = 0 + scorta disponibile → "stabile" 0ms
  ✓ U > 0 ma ≤ soglia_warn → "attenzione" 0ms
  ✓ U > soglia_warn → "urgente" 0ms
✓ quantitaSuggestita (2)
  ✓ consumo=0.5 unità/min, t_riapp=600s (10min) → ceil(0.5 × 10 × 1.3) = 7 0ms
  ✓ consumo=2 unità/min, t_riapp=900s (15min) → ceil(2 × 15 × 1.3) = 39 0ms
✓ aggregaConsumiCondivisi (1)
  ✓ voci [1,2] condivise → entrambe vedono la somma dei consumi 0ms
✓ tempoResiduoEventoSec (2)
  ✓ orario futuro nello stesso giorno → secondi positivi 0ms
  ✓ orario passato → 0 0ms
✓ derivaStatoVisivo (5)
  ✓ scorta non attiva → "sconosciuto" 0ms
  ✓ quantita = 0 → "esaurito" 0ms
  ✓ quantita ≤ soglia_rosso → "critico" 0ms
  ✓ quantita ≤ soglia_giallo → "attenzione" 0ms
  ✓ quantita sopra tutte le soglie → "disponibile" 0ms
✓ valutaVoce: pipeline completa (2)
  ✓ voce senza consumo → stabile con tempo_esaurimento null 0ms
  ✓ voce con quantita=0 → esaurito 0ms
✓ eseguiPredizione (mock database) (3)
  ✓ calcola la predizione aggregando i dati dal mock del repository 1ms
  ✓ calcola la predizione per una singola voce (eseguiPredizionePerVoce) 1ms
  ✓ gestisce indici condivisi (tabella voce_contatore) e catch errori 1ms

```

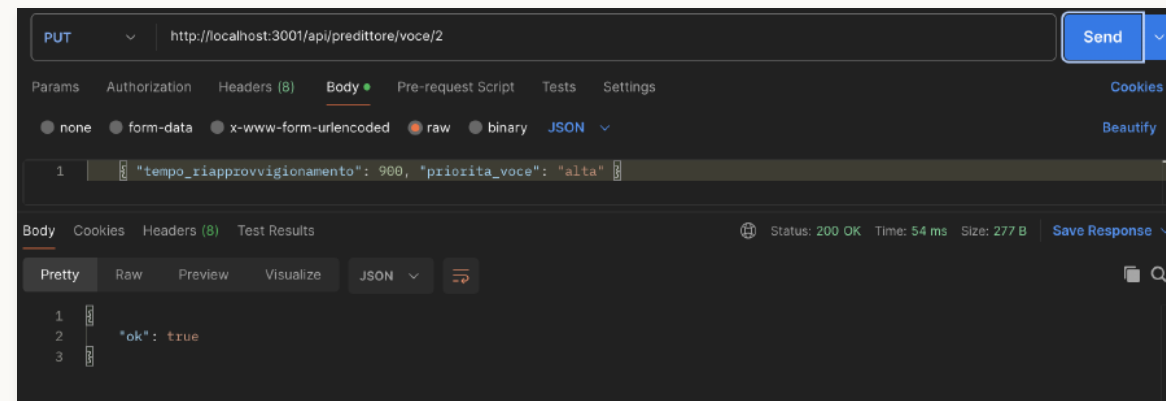
33/33 test unitari superati - calcolaPesi, consumoAtteso, calcolaU e classificazione scorte

API Predittore - Test Postman

51

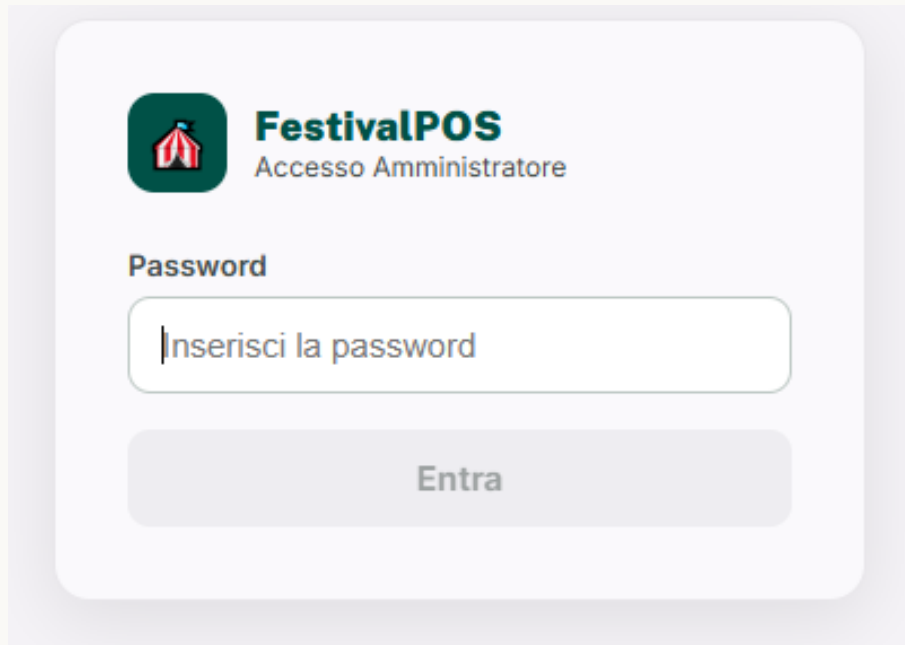


PUT configurazione - parametro evento_fine_oggi



PUT voce/:id - priorità e tempo di riapprovvigionamento

Iterazione 3 - Admin (Login)



The image shows a login form for 'FestivalPOS'. At the top left is a logo of a red and white striped tent inside a green rounded square. To its right, the text 'FestivalPOS' is in bold green, and 'Accesso Amministratore' is in a smaller, lighter green font below it. Below the logo and text, the word 'Password' is written in bold. Underneath is a white input field with a light gray border and rounded corners, containing the placeholder text 'Inserisci la password'. Below the input field is a wide, light gray button with rounded corners and the text 'Entra' in a medium gray font.

Password : hash bcrypt con salt per-hash. Token opaco con scadenza, volatile in memoria. Middleware richiediAdmin su tutte le rotte admin.

Iterazione 3 - Predittore Scorte

 **Predittore Scorte**

Aggiornato alle 17:04:02

← Torna alla Cassa

Tutti 25Urgenti 1Attenzione 1Esauriti 1Stabili 22

Reparto: TuttiRefresh: 30sAggiorna

STATO	VOCE	REPARTO	PRIORITÀ	QTY	C 15MIN	C 1H	STORICO/H	MODALITÀ	T ESAUR.	U	SUGGERIMENTO
ESAUrito	Acqua B002	bar	media	0	10	10	0.0 (0)	prudente	0s	720.0	—
URGENTE	Risotto ai funghi P001	cucina	media	2	8	8	0.0 (0)	prudente	5m	351.2	+6
ATTENZIONE	Lasagne al forno P003	cucina	media	3	7	7	0.0 (0)	prudente	8m	87.8	—
STABILE	Polenta taragna P002	cucina	media	10	0	0	0.0 (0)	prudente	∞	0.0	—
STABILE	Pasta al pomodoro P004	cucina	media	9	1	1	0.0 (0)	prudente	3.1h	0.0	—
STABILE	Gnocchi al ragu P005	cucina	media	9	1	1	0.0 (0)	prudente	3.1h	0.0	—
STABILE	Penne all'arrabbiata P006	cucina	media	10	0	0	0.0 (0)	prudente	∞	0.0	—
STABILE	Costine alla griglia S001	griglia	media	9	1	1	0.0 (0)	prudente	3.1h	0.0	—
STABILE	Salsiccia alla piastra S002	griglia	media	10	0	0	0.0 (0)	prudente	∞	0.0	—
STABILE	Pollo arrosto S003	cucina	media	9	1	1	0.0 (0)	prudente	3.1h	0.0	—

Risultati e sviluppi futuri

OBIETTIVI RAGGIUNTI

- Sistema POS completo e funzionante in 3 iterazioni
- Smistatore intelligente con load balancing dinamico
- Predittore dinamico proattivo per il riordino scorte
- 94 test totali: copertura branch 94.4%, linee 100%

SISTEMA POS PER FESTE E SAGRE DI PAESE

55

Quick
Byte

Grazie

Luca Maccari
Matr. 1081951

Gabriele Masinari
Matr. 1079692

Francesco Tomasoni
Matr. 1080980